INSTITUTO FEDERAL DE SÃO PAULO

Sabor e Clic

Como uma reserva vira atendimento, pedido e baixa de estoque

Projeto Integrador / 2026

Arquitetura do atendimento

O navegador envia JSON à API Flask. O servidor aplica as regras de negócio e grava o resultado no MySQL.

Interface

As páginas HTML carregam JavaScript para renderizar listas e formulários. domain.js representa pratos, itens, carrinho e pedido. views.js coordena ações da interface.

API Flask

api_routes.py registra rotas. Os handlers convertem entradas HTTP em operações. Os controllers validam a identidade, os dados e as transições antes de persistir.

Persistência

Os models representam as tabelas. No checkout, a gravação do pedido, itens e movimentações compartilha a mesma transação.

Perfis e permissões

Cada perfil tem telas e operações próprias. A API verifica o papel e, para clientes, o vínculo com os registros.

Cliente	Consulta bancadas, faz e acompanha suas reservas, inicia o atendimento no horário contratado, monta o carrinho e acompanha seus pedidos.
Cozinha	Consulta a fila no KDS, informa a estimativa e avança o pedido na sequência operacional. Pode alterar a disponibilidade de pratos.
Administrador	Mantém bancadas, usuários, pratos, ingredientes, fichas técnicas e substituições. Acompanha reservas, movimentações, relatórios e auditoria.

Conflitos de reserva

A API bloqueia a linha da bancada no MySQL antes de verificar se já existe uma reserva conflitante.

operacao_controller.py / linhas 54–63

```
54      end = date(data.get('fim', record.fim if record else None), 'fim')
55      if end <= start or start < now():
56          fail('Informe um período futuro com fim posterior ao início.')
57      query = select(m.Reserva).where(m.Reserva.id_bancada == bid,
58          m.Reserva.status.in_(('confirmada', 'em_andamento')),
59          m.Reserva.inicio < end, m.Reserva.fim > start)
60      if record:
61          query = query.where(m.Reserva.id_reserva != record.id_reserva)
62      if db.session.scalar(query.with_for_update()):
63          fail('Esta bancada já está reservada neste período.', 409)
```

Período

Bancada ativa, início no futuro e fim após o início. Sobreposição ocorre se uma reserva vigente começa antes do fim solicitado e termina após o início solicitado.

Exemplo

Uma reserva 14h–15h impede 14h30–15h30 na mesma bancada (HTTP 409). Uma reserva a partir das 15h pode seguir se as demais condições forem atendidas.

Tarifa

Com uma tarifa hipotética de R$ 40 por hora, um período de 90 minutos custa R$ 60. O servidor calcula a partir da duração em segundos e grava o total.

Check-in e sessão

O check-in vincula uma sessão à reserva. A janela válida começa no início da reserva e termina imediatamente antes do fim.

operacao_controller.py / linhas 83–90

```
83      reservation = owned(get(m.Reserva, integer(data.get('id_reserva'),
         'id_reserva'), lock=True))
84      if reservation.status != 'confirmada':
85          fail('A reserva não está confirmada.', 409)
86      if now() >= reservation.fim:
87          fail('O período desta reserva já terminou.', 409)
88      if now() < reservation.inicio:
89          fail('Aguarde o horário de início da reserva.', 409)
90      if db.session.scalar(select(m.Sessao).where(m.Sessao.id_reserva ==
         reservation.id_reserva)):
```

Abertura

O servidor bloqueia a reserva e verifica a propriedade, o status confirmado, o horário e a existência de sessão anterior. A reserva passa para em_andamento.

Seleção

Com atendimentos em bancadas distintas, o cliente escolhe a sessão à qual o próximo pedido será associado.

Encerramento

O fim do horário impede novos pedidos. O cliente encerra a sessão manualmente após todos os pedidos ficarem entregues ou cancelados. A reserva passa a concluída.

Carrinho e envio do pedido

O carrinho apresenta uma estimativa. A confirmação usa o preço cadastrado no servidor.

views.js / linhas 190–196

```
190      const storageKey='pending-order:'+currentUser.id_usuario;
191      let pending;
192      try{pending=JSON.parse(localStorage.getItem(storageKey)||'null');}
         catch{pending=null;}
193      const
         key=pending?.assinatura===order.assinatura?pending.chave:order.cha
         ve;
194      localStorage.setItem(storageKey,JSON.stringify({assinatura:order.a
         ssinatura,chave:key}));
195      await api('/pedidos',{method:'POST',headers:{'Idempotency-
         Key':key},body:order.toPayload()});
196      localStorage.removeItem(storageKey);
```

No navegador

Carrinho trabalha em centavos. Pedido gera um payload com identificadores, quantidades, endereço e observações. A tela persiste a chave de envio antes de chamar a API.

Na API

POST /api/pedidos exige uma sessão do cliente, ativa e dentro do horário. Quantidades repetidas do mesmo prato são somadas e os pratos ativos são consultados no banco.

Exemplo

Se o catálogo gravado no servidor tem preço de R$ 32,50, duas unidades totalizam R$ 65,00. O item conserva nome e preço unitário para o histórico.

Reenvio após falha de rede

Se o pedido foi gravado e a resposta se perdeu, repetir o conteúdo com a mesma chave recupera o pedido existente.

operacao_controller.py / linhas 147–151

```
147      digest=sha256(json.dumps([session.id_sessao,sorted(counts.items())
         ,address,notes],ensure_ascii=False).encode()).hexdigest()
148      previous=db.session.scalar(select(m.Pedido).where(m.Pedido.id_sess
         ao==session.id_sessao,m.Pedido.chave_idempotencia==key))
149      if previous:
150          if previous.hash_requisicao!=digest:fail('Esta chave já foi
         usada para outro conteúdo.',409)
151          return previous
```

Identidade do envio

A chave é validada e pesquisada dentro da sessão. O hash inclui a sessão, quantidades por prato ordenadas, endereço e observações.

Mesmo conteúdo

O servidor retorna o pedido encontrado antes de passar pela alocação de estoque. A confirmação já gravada permanece única.

Conteúdo alterado

A mesma chave com quantidades ou observações diferentes gera HTTP 409. Uma nova compra usa uma nova chave.

Ficha técnica e substituição

A baixa considera a quantidade total de cada ingrediente no pedido, somando a necessidade de todos os pratos.

estoque_controller.py / linhas 45–53

```
45          for alternative in alternatives:
46              if alternative.id_ingrediente_original!=original:continue
47              amount=(required*alternative.fator_conversao).quantize(Dec
        imal('.001'),rounding=ROUND_CEILING)
48              substitute=alternative.id_ingrediente_substituto
49              if available.get(substitute,0)>=amount:
50                  available[substitute]-
        =amount;result.append((substitute,original,amount));break
51          else:fail(f'Estoque insuficiente para o ingrediente
        #{original} e seus substitutos.',409)
52      return result
53
```

Agregação

Duas porções com 0,150 kg por porção exigem 0,300 kg. Se outro prato do pedido usa 0,100 kg do mesmo ingrediente, a necessidade sobe para 0,400 kg.

Alocação

O sistema bloqueia os saldos envolvidos e reserva os originais com saldo suficiente antes de procurar alternativas. O substituto direto precisa atender a necessidade inteira, segundo prioridade, fator e saldo.

Exemplo

Para 0,400 kg e fator 1,25, o substituto precisa de 0,500 kg. A movimentação identifica o ingrediente efetivamente debitado e o original da receita.

Gravação e baixa de estoque

O checkout confirma o pedido, os itens e as movimentações em um único commit.

operacao_controller.py / linhas 170–177

```
170        db.session.add(order)
171        db.session.flush()
172        for dish, quantity, subtotal in prepared:
173            db.session.add(m.ItemPedido(id_pedido=order.id_pedido,
           id_prato=dish.id_prato,
174                    quantidade=quantity, preco_unitario=dish.preco,
           subtotal=subtotal,nome_prato=dish.nome))
175        db.session.flush()
176        consume(order.id_pedido,allocations)
177        db.session.commit()
```

Ordem da gravação

O servidor prepara valores, calcula a alocação e obtém o ID do pedido com flush. Em seguida, inclui os itens e aplica o consumo.

Garantia de saldo

A atualização do saldo exige que a quantidade resultante seja maior ou igual a zero. A movimentação registra a quantidade assinada e uma chave única por evento.

Falha durante a operação

Quando falta saldo ou outra validação falha, o tratamento de erro desfaz as alterações pendentes. O controle do estoque ocorre no backend, conforme a decisão de arquitetura do projeto.

Fila da cozinha e cancelamento

O KDS acompanha pedidos ativos. A cozinha informa a estimativa e registra o avanço de cada ticket.

Sequência	Recebido, em preparo, pronto e entregue. O servidor valida cada avanço e grava a data da transição. Tentar pular de recebido para pronto retorna HTTP 409.
Cancelamento	O administrador pode cancelar um pedido ainda ativo. Em recebido, o sistema estorna o ingrediente efetivamente consumido na confirmação.
Após iniciar preparo	O cancelamento encerra o pedido e o saldo permanece debitado. Se houver correção física, ela fica registrada em uma nova movimentação.

O pedido no KDS

O ticket mostra a bancada, os itens, as observações e o estado atual do preparo.



Em preparo

O pedido #1 está na bancada Toscana, tem 2 risotos e exibe estimativa de 8 minutos. A próxima ação é Marcar pronto.

Captura do ambiente descartável / output/evidencias/cozinha-kds.png

Autenticação e autorização

O login cria uma sessão autenticada revogável. Essa sessão identifica a conta, enquanto a sessão de atendimento vincula pedidos à reserva.

api.js / linhas 2–13

```
2  let csrfToken = '';
3  async function api(path, options = {}) {
4    const headers = {...options.headers};
5    if (options.body !== undefined) headers['Content-Type'] =
       'application/json';
6    if (csrfToken) headers['X-CSRF-Token'] = csrfToken;
7    let response;
8    try {
9      response = await fetch((window.SABOR_API_BASE || '/api') + path, {
10       ...options, headers, credentials: 'include',
11       body: options.body === undefined ? undefined :
       JSON.stringify(options.body)
12     });
13   } catch {
```

Identidade

O servidor verifica a senha por hash. As sessões têm duração absoluta de oito horas e expiração por 30 minutos de inatividade. Logout revoga a sessão.

Escritas

O navegador envia o cookie e o cabeçalho X-CSRF-Token. A API valida o token nas operações protegidas e confere papel e propriedade.

Rastreabilidade

O login tem limites por conta e IP. Eventos de auditoria identificam ação, recurso, resultado e correlação sem incluir senha, cookie ou token.

Dados, consultas e operação

Reserva, sessão, pedido e item preservam a trajetória do atendimento. A ficha técnica e as movimentações explicam a trajetória do estoque.

Relacionamentos

A bancada recebe várias reservas. Cada reserva pode originar uma sessão. Uma sessão pode conter vários pedidos. Cada pedido grava seus itens e pode registrar várias movimentações.

Consultas

Listas usam paginação e filtros. O resumo de sessão agrega pedidos e consumo no servidor. Relatórios filtram por período e mostram receitas, ticket médio e tempo de preparo.

Instalação

Em banco existente, execute flask --app run init-db com conexão administrativa, conforme o README. Volte às credenciais de runtime, reinicie o Flask e valide o fluxo. A restauração exige um esquema inicializado e vazio.

Demonstração e evidências

Um percurso em ambiente descartável permite apresentar a operação sem usar dados reais.

Reserva e check-in

Cadastre ou selecione uma bancada ativa. Crie uma reserva futura, tente sobrepor o período e mostre o conflito. Para o checkout, use uma sessão no período válido ou a fixture.

Cliente e cozinha

Envie um pedido com duas unidades. Mostre o ticket no KDS, informe a estimativa, avance os estados, volte ao cliente e encerre o atendimento após a entrega.

Verificações registradas

Em 02/11/2026, o projeto registrou 41 testes Python e 8 JavaScript, além de testes de concorrência de reservas e estoque no MySQL isolado. As capturas são do ambiente descartável. A verificação da instalação real continua registrada como pendente.